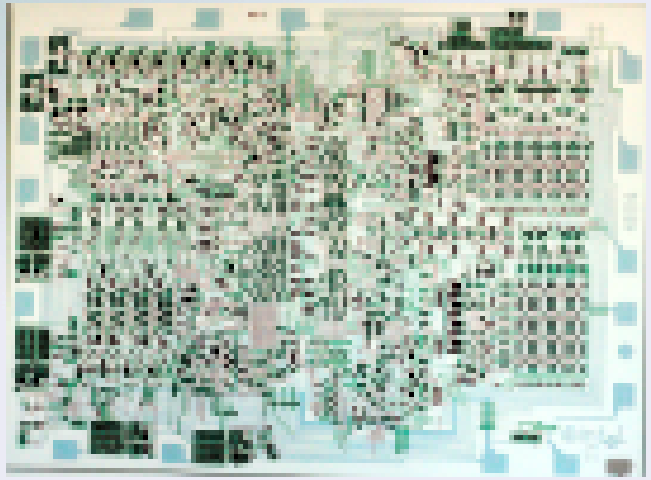


Lecture 7b: Recursion and Stacks

CSCI11: **Computer Architecture and Organization**

Readings

- *Chapter 8.1-8.2* Patt and Patel: Introduction to Computing Systems...
- *Guide to Using LC-3 Tools* - *Instructor Repo*
- *Appendix A* Patt and Patel: Introduction to Computing Systems... - *Instructor Repo*

Recursion is when a function calls itself to solve a problem by breaking it into smaller, similar subproblems.

Key Idea

Instead of using loops, we solve problems by:

- Taking one small piece of the problem
- Solving the rest of the problem the same way
- Combining the results

Simplifies by Reduction

By reducing a problem into the most simple concept, the problem can be solved by continuing to reduce until a defined end is met, **tree transversal** is good example

Tree Transversal - hierarchical structures in programming

- file systems
- HTML
- networks

As well as:

- **Divide and conquer** algorithms (sorting, searching)
- **Mathematical problems** (factorials, Fibonacci)
- **Graph traversal** family trees (biology)

Recursion is Natural For:

- Problems that are defined in terms of themselves
- Elegant, readable solutions
- Thinking about problems differently

When NOT to Use Recursion:

- Simple counting loops (use `for` loops instead)
- Very deep recursion (can crash with stack overflow)
- Performance-critical code (loops are faster)

The Two Essential Parts of Recursion

1. Base Case (The Stopping Point)

```
# Without this, recursion never stops!
if condition_is_met:
    return some_value  # Stop here, don't call again
```

2. Recursive Case (The Self-Call)

```
# Call the function again with a simpler problem
else:
    return something + function(simpler_problem)
```

Critical Rule

Each recursive call MUST move closer to the base case, or your program will crash

Example 1: Countdown

Simple Recursion in Action

```
def countdown(n):
    # BASE CASE: Stop when we reach 0
    if n == 0:
        print("Blastoff! 🚀")
        return

    # RECURSIVE CASE: Print and call again with n-1
    print(n)
    countdown(n - 1)

countdown(5)
```

Output:

```
5
4
3
2
1
Blastoff! 🚀
```

What Happens:

- `countdown(5)` prints 5, then calls `countdown(4)`
- `countdown(4)` prints 4, then calls `countdown(3)`
- ... continues until `countdown(0)` hits the base case

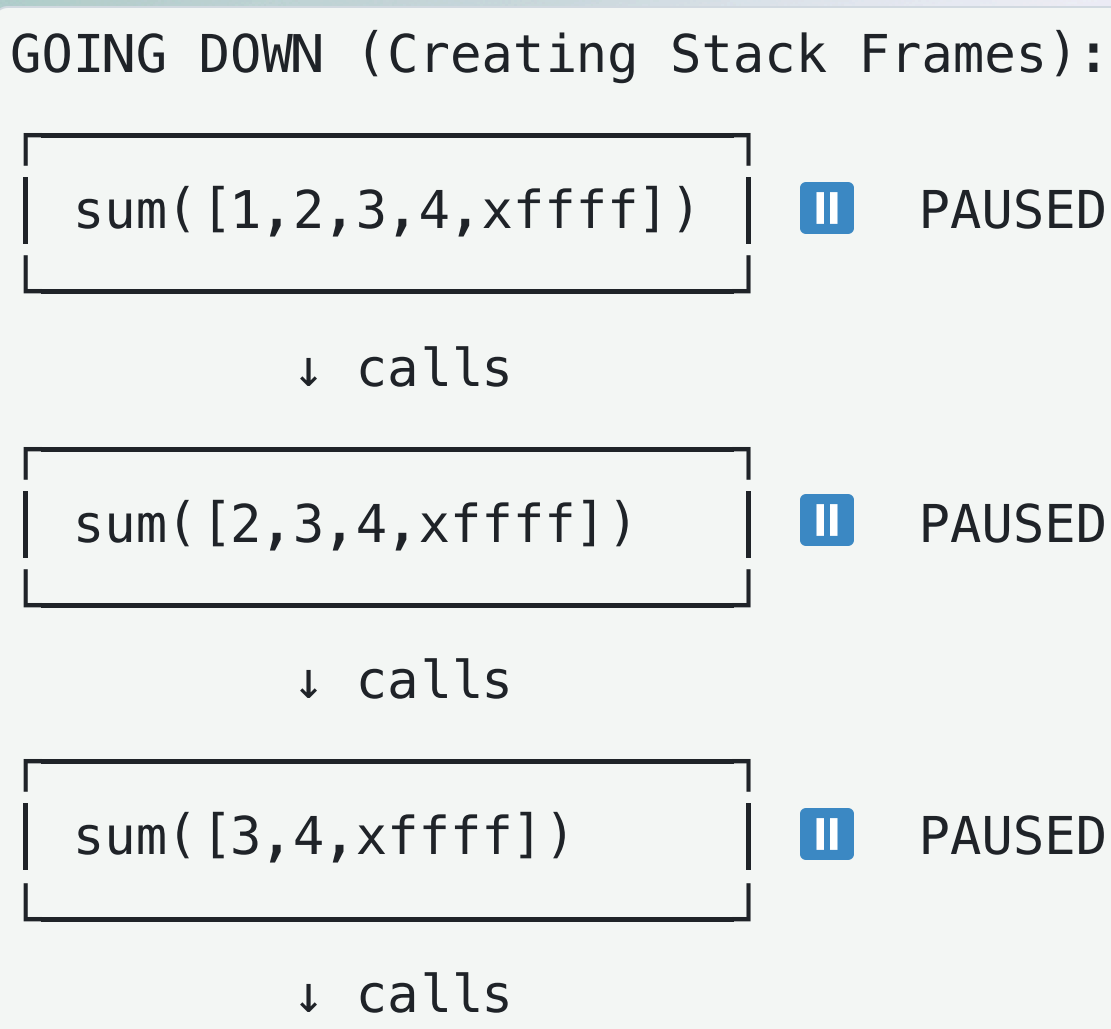
What happens if you switch the print command and the recursive call?

LC3 - Recursive Program sum_r.asm

```
; Description: Recursive sum of an integer list, using sentinel xFFFF to
;              mark the end of the list – the same pattern as .STRINGZ
;              uses x0000 to mark the end of a string.
```

Review sum_r.asm

The Call Stack Explained



GOING UP (Returning Values):
Base case returns R0 = 0
Previous call: R0 + 4 = 4
Previous call: R0 + 3 = 7
Previous call: R0 + 2 = 9
Previous call: R0 + 1 = 10
Stores result in SUM

Key Insight:

- Each call **pauses** and waits for the recursive call to finish
- When the base case is reached, values **return back up** the stack
- Each level uses the returned value to complete its calculation
- **The "work" is performed on the return, not the call**